Recovering Reasoning Fidelity in Quantized Small Language Models Through Inference-Time Techniques

Shaswat Ria Munshi Kanishka Gupta Avni Tongia

Deep Learning, BITS Pilani, April 2026

Abstract

Quantization reduces model size by representing weights in fewer bits, but its effect on multi-step mathematical reasoning in sub-3B models is not well understood. We study how weight quantization (BF16 → W8A16 → W4A16) affects reasoning in Qwen2 1.5B and LLaMA 1B on GSM8K and MATH, and evaluate three inference-time recovery methods: greedy decoding, self-consistency sampling ($N = 5$), and speculative decoding which intervene at different points in the generation process. We additionally fine-tune the quantized Qwen2 1.5B using GRPO to establish a training-time upper bound and quantify the gap between what inference-time methods can recover and what retraining (through GRPO) achieves.

Our main findings are: (1) quantization effects are not uniform. Qwen2 drops from 66.0% to 40.0% on MATH; LLaMA remains robust on GSM8K despite similar compression; (2) self-consistency hurts Qwen2 at W4A16 but consistently helps LLaMA, suggesting model-dependent error structure under aggressive compression; (3) speculative decoding achieves 54.0% at W4A16 versus 38.0% for self-consistency and 50.0% for greedy, confirming that during-generation verification is more robust than post-generation aggregation when errors become systematic; (4) cascade analysis shows early arithmetic errors that doom final answers increase from 45.5% to 70.0% on Qwen2 MATH under W4A16, explaining mechanistically why recovery becomes harder under aggressive compression; (5) GRPO achieves 64.0% versus 54.0% for the best inference-time method - a gap of 10 percentage points.

1 Introduction

Large language models (LLMs) have shown strong performance on reasoning tasks, but deploying them on resource-constrained devices such as laptops, phones, and embedded systems is difficult because of their high memory and compute requirements. Quantization addresses this by representing model weights in fewer bits—for example, 8-bit or 4-bit integers instead of the standard 16-bit floating point—which reduces model size and speeds up inference without changing the model architecture.

Prior work has shown that quantization can preserve performance on general natural language tasks reasonably well: Dettmers et al. [1] demonstrated that 4-bit quantization with LoRA adapters maintains quality across a range of language benchmarks, Wang et al. [2] showed that even 1-bit weight quantization is feasible with appropriate training-time design, and Jin et al. [3] found that 4-bit quantization generally maintains performance comparable to full precision on instruction-following benchmarks. However, these findings are largely on general language tasks rather than multi-step mathematical reasoning, where the sequential nature of the task makes it more sensitive to small weight perturbations. Studies also show how quantization affects final answer accuracy on mathematical reasoning tasks [4, 5], but less attention has been paid to where in the reasoning chain errors first appear, whether those errors cascade, and whether inference-time techniques can recover lost performance without retraining the model.

This paper studies these questions on two small language models: Qwen2 1.5B and LLaMA 1B evaluated on GSM8K [6] and MATH, across three quantization levels: BF16 (full precision

baseline), W8A16 (8-bit weights), and W4A16 (4-bit weights). We evaluate three inference-time recovery methods that require no changes to model weights, each intervening at a different point in the generation process. Greedy decoding serves as the deterministic baseline. Self-consistency sampling [7] generates $N = 5$ independent reasoning paths and selects the most common final answer by majority voting—a post-generation aggregation approach. Speculative decoding [8] uses the W4A16 model as a fast draft and a higher-precision model as a verifier that checks each token during generation—a during-generation verification approach. We also fine-tune the quantized Qwen2 1.5B using Group Relative Policy Optimization (GRPO) [5] to establish a training-time upper bound and quantify the gap between what inference-time methods can recover and what retraining achieves.

Our main findings are: 1) quantization effects are model and dataset dependent; 2) self-consistency helps at moderate compression but actively hurts Qwen2 at W4A16 while LLaMA remains unaffected; 3) speculative decoding outperforms both greedy and self-consistency at W4A16, confirming that during-generation verification is more robust than post-generation aggregation when errors become systematic; 4) cascade analysis provides a mechanistic explanation for why recovery becomes harder under aggressive compression; and 5) GRPO establishes a training-time ceiling 10 percentage points above the best inference-time method. Full results with numbers are reported in Section 5.

This paper provides a thorough empirical study of quantization effects on reasoning in small language models. We begin by characterising how quantization degrades reasoning at the step level rather than just the final answer, followed by an evaluation of inference-time recovery methods operating at different intervention points, and conclude with a comparison against a training-time upper bound established through GRPO fine-tuning.

2 Related Work

2.1 Quantization Methods

Quantization reduces the memory footprint of language models by storing weights in lower bit-width formats. Dettmers et al. [1] introduced QLoRA, which combines 4-bit NormalFloat quantization with LoRA adapters to enable fine-tuning of large models on consumer hardware without significant performance loss. Wang et al. [2] proposed BitNet, which trains transformer models from scratch with weights constrained to $\{-1, +1\}$, demonstrating that extreme 1-bit quantization is feasible when applied during training rather than after. Jin et al. [3] conducted a broad evaluation of quantization strategies across multiple language benchmarks, finding that 4-bit quantization generally maintains performance comparable to full precision on instruction-following tasks, while identifying outlier weight isolation as important for more aggressive 2-bit quantization.

2.2 Quantization and Mathematical Reasoning

While quantization works reasonably well for general language tasks, its effect on mathematical reasoning is less well understood. Li et al. [4] studied low-bit quantization degradation specifically for mathematical reasoning, evaluating methods such as AWQ, GPTQ, and SmoothQuant across different bit settings. They proposed a seven-dimensional error taxonomy covering logical errors, computation errors, step omission, and symbol misuse, moving beyond final-answer evaluation to diagnose how quantization disrupts specific reasoning capabilities. Liu et al. [5] conducted a large-scale empirical study of how weight and activation quantization affects reasoning models ranging from 1.5B to 70B parameters across mathematics, science, and coding benchmarks, finding that moderate quantization preserves reasoning but aggressive low-bit settings degrade performance, particularly on harder tasks and smaller models.

2.3 Inference-Time Recovery

Several methods have been proposed to improve reasoning accuracy at inference time without modifying model weights. Wang et al. [7] introduced self-consistency, which generates multiple independent reasoning paths and selects the most common final answer by majority voting. This method improves accuracy on reasoning benchmarks without any additional training. Zhao et al. [8] proposed QSPEC, a speculative decoding approach that uses a fast low-precision draft model and a higher-precision verifier to maintain reasoning quality while reducing latency. These methods have been studied in the context of full-precision models, but their behaviour under quantization particularly whether they become less effective as compression increases has not been systematically studied.

2.4 Reinforcement Learning for Reasoning

Reinforcement learning has been applied to improve the reasoning ability of language models. Group Relative Policy Optimization (GRPO), introduced in DeepSeek-R1 [9], trains the model by generating groups of responses, scoring them with a reward signal, and updating weights to reinforce above-average responses without requiring a separate critic model. In our work, we apply GRPO to an already-quantized model using binary correctness as the reward signal, treating the result as a training-time upper bound rather than a primary contribution.

2.5 Summary of Gaps

Existing work predominantly evaluates models of 7B parameters and above, focuses on final answer accuracy rather than step-level reasoning quality, and has not studied how inference-time recovery methods behave as a function of quantization level. Our work addresses all three of these gaps on a controlled set of problems using two model families and two datasets.

3 Methodology

3.1 Models

We evaluate two small language model families. The primary model is Qwen2 1.5B Instruct, a 1.5 billion parameter instruction-tuned model. The secondary model is LLaMA 1B Instruct, a 1 billion parameter instruction-tuned model.

3.2 Quantization Configurations

We evaluate three precision levels for each model, summarised in Table 1.

Table 1: Quantization configurations used in this work.

Config	Weight Bits	Activation Bits	Method
BF16	16 (bfloat16)	16	No quantization — full precision baseline
W8A16	8 (INT8)	16	`bitsandbytes load_in_8bit=True`
W4A16	4 (NF4)	16	`bitsandbytes load_in_4bit=True, bnb_4bit_quant_type="nf4"`

This progression moves from no compression to moderate compression to aggressive compression, allowing us to observe how reasoning quality changes across bit-widths.

3.3 Datasets

We use two mathematical reasoning benchmarks. GSM8K [6] is a dataset of grade school mathematics problems that require multi-step arithmetic reasoning. We use 50 problems from

the test split, stratified by difficulty into 20 easy, 20 medium, and 10 hard problems based on the number of arithmetic steps in the gold solution. MATH is a competition-level mathematics dataset with significantly harder problems and longer reasoning chains. We use 50 problems stratified by difficulty using MATH's own difficulty levels: Level 1 as easy, Level 3 as medium, and Level 5 as hard. The same 50 GSM8K problem IDs are used for both Qwen2 1.5B and LLaMA 1B to ensure cross-model comparisons are valid.

3.4 Inference-Time Recovery Methods

We evaluate three inference-time methods. Greedy decoding serves as the deterministic baseline with no recovery applied.

3.4.1 Self-Consistency Sampling

Following Wang et al. [7], we generate $N = 5$ independent reasoning paths at temperature 0.7 and select the final answer by majority voting. This method operates entirely after generation is complete as a post-generation aggregation approach. It is most effective when errors across different sampling paths are independent, so that the correct answer wins by majority. When errors become correlated across paths under aggressive compression, voting can amplify the wrong answer rather than correct it.

3.4.2 Speculative Decoding

Following Zhao et al. [8], we use the W4A16 quantized model as a fast draft model and a higher-precision model (W8A16 or BF16) as a verifier. Unlike self-consistency which intervenes after generation, speculative decoding intervenes during generation. The draft model proposes tokens one at a time and the verifier accepts or rejects each token before the next one is generated. We use HuggingFace's built-in `assistant_model` parameter which handles token-level acceptance and rejection automatically and evaluate two verifier configurations: W4A16 draft with W8A16 verifier, and W4A16 draft with BF16 verifier. This method uses our existing quantization tiers and requires no additional models beyond what was already used in the greedy experiments.

3.5 Training-Time Upper Bound: GRPO

To establish how much accuracy a training-time approach can recover, we fine-tune the quantized Qwen2 1.5B W8A16 model using Group Relative Policy Optimization (GRPO). We attach LoRA adapters [1] to the quantized model so that gradients can flow through the frozen quantized weights via the adapters. The reward signal is binary: 1.0 if the extracted final answer matches the ground truth and 0.0 otherwise. We use group size $G = 4$, meaning the model generates 4 responses per prompt and is updated to reinforce responses that scored above the group average. Training runs for one epoch on 200 problems sampled from the GSM8K training split using seed 42. We evaluate the fine-tuned model on the same 50 test problems used for all other experiments.

3.6 Evaluation Metrics

We report the following metrics for each configuration.

Table 2: Evaluation metrics used in this work.

Metric	Description	Metric	Description
Accuracy	Percentage of problems where the extracted final answer matches ground truth. Reported overall and by difficulty level.	SC Effect	Problem-level classification of how self-consistency changes outcomes relative to greedy: SC Helped, SC Hurt, Both Correct, or Both Wrong.
First Error Step	Step number where the first arithmetic mistake appears in the reasoning chain. Computed on wrong answers only. Logical errors are not detected.	Problem Consistency	Classification of each problem across bit-widths: Robust, Fragile at W4A16, Fragile at W8A16, Improved by Quantization, or Always Wrong.
Cascade Rate	Proportion of wrong answers with a detected first error step. A lower bound on the true cascade rate since only arithmetic errors are detected.		

4 Experimental Setup

4.1 Hardware and Software

All experiments were run on an NVIDIA T4 GPU via Google Colab free tier. Quantization was implemented using the bitsandbytes library. GRPO fine-tuning used the TRL library version 1.2.0 with the GRPOTrainer class. Model loading and inference used the HuggingFace Transformers library, LoRA adapters via the PEFT library, and speculative decoding via HuggingFace's built-in `assistant_model` parameter.

4.2 Decoding Settings

Greedy decoding (`do_sample=False`) was used as the baseline for all quantization configurations. Self-consistency used temperature 0.7 with $N = 5$ samples per problem. Speculative decoding also used greedy decoding, with the draft model proposing tokens and the verifier accepting or rejecting each one. All experiments used a single random seed (42) due to compute constraints.

4.3 Prompt Format

All models were prompted using their respective Instruct chat templates. Each prompt instructed the model to show its reasoning step by step and end with the final answer in `\boxed{}`. Answer extraction used regex-based parsing that looked for boxed answers first, then the GSM8K `####` format, and finally the last number in the output.

4.4 Inference Experiment Details

Table 3 summarises the configurations used across all inference experiments.

Table 3: Inference experiment configurations. SC = self-consistency ($N = 5$). Spec Dec = speculative decoding. MATH experiments used greedy only due to session time constraints except for LLaMA which ran greedy and SC.

Model	Dataset	Quant	Strategy	Problems	Approx. Time
Qwen2 1.5B	GSM8K	BF16	Greedy + SC	50	∼90 min
Qwen2 1.5B	GSM8K	W8A16	Greedy + SC	50	∼90 min
Qwen2 1.5B	GSM8K	W4A16	Greedy + SC	50	∼90 min
Qwen2 1.5B	MATH	BF16	Greedy	50	∼120 min
Qwen2 1.5B	MATH	W8A16	Greedy	50	∼120 min
Qwen2 1.5B	MATH	W4A16	Greedy	50	∼120 min
LLaMA 1B	GSM8K	BF16	Greedy + SC	50	∼60 min
LLaMA 1B	GSM8K	W8A16	Greedy + SC	50	∼60 min
LLaMA 1B	GSM8K	W4A16	Greedy + SC	50	∼60 min
LLaMA 1B	MATH	BF16	Greedy + SC	50	∼90 min
LLaMA 1B	MATH	W8A16	Greedy + SC	50	∼90 min
LLaMA 1B	MATH	W4A16	Greedy + SC	50	∼90 min
Qwen2 1.5B	GSM8K	W4A16	Spec Dec (W8A16 verifier)	50	∼75 min
Qwen2 1.5B	GSM8K	W4A16	Spec Dec (BF16 verifier)	50	∼55 min

4.5 GRPO Training Details

Table 4 summarises the GRPO fine-tuning configuration.

Table 4: GRPO training configuration.

Parameter	Value
Base model	Qwen2 1.5B Instruct
Quantization	INT8 (W8A16)
LoRA rank (r)	8
LoRA alpha (α)	32
LoRA dropout	0.05
LoRA target modules	`q_proj`, `v_proj`
Group size (G)	4
Learning rate	1×10^{-5}
Epochs	1
Batch size	1 (gradient accumulation steps: 4)
Max new tokens	1024
Training problems	200 (GSM8K train split, seed 42)
Training time	∼175 minutes
Peak VRAM	8.87 GB

4.6 Scope Reduction

Due to Colab free tier session length constraints (∼4–5 hours), all experiments use 50 problems per dataset rather than the full test sets (1,319 for GSM8K; the full MATH set), with a single random seed. MATH problems generate reasoning chains 3–5 times longer than GSM8K, so self-consistency on Qwen2 MATH would have exceeded session limits and was omitted. Speculative decoding was run on Qwen2 GSM8K only, and GRPO training was limited to 200 problems. These constraints are acknowledged as limitations; results on larger evaluation sets may differ.

5 Results

5.1 Overall Comparison

Table 5 and Table 6 summarise accuracy across all configurations on GSM8K and MATH respectively.

Table 5: GSM8K accuracy (%) across all strategies and bit-widths. SC = self-consistency ($N = 5$). Spec Dec = speculative decoding (W4A16 draft only). GRPO applied to Qwen2 W8A16 only. Best result per model shown in **bold**.

		Qwen2 1.5B			**LLaMA 1B**		
Strategy	**Verifier**	BF16	W8A16	W4A16	BF16	W8A16	W4A16
Greedy	—	42.0	46.0	50.0	68.0	72.0	66.0
SC ($N = 5$)	—	56.0	52.0	38.0	82.0	82.0	**84.0**
Spec Dec	W8A16	—	—	54.0	—	—	—
Spec Dec	BF16	—	—	54.0	—	—	—
GRPO	—	—	**64.0**	—	—	—	—

Table 6: MATH accuracy (%) across strategies and bit-widths. Qwen2 used greedy only (base model, train split). LLaMA used greedy and SC (instruct model, test split). Direct cross-model comparison is limited — see Section 5.4. Best result per model shown in **bold**.

	Qwen2 1.5B			**LLaMA 1B**		
Strategy	BF16	W8A16	W4A16	BF16	W8A16	W4A16
Greedy	**66.0**	62.0	40.0	32.0	30.0	24.0
SC ($N = 5$)	—	—	—	**40.0**	34.0	28.0

5.2 GSM8K Results

Qwen2 1.5B greedy accuracy does not drop monotonically with bit-width on this 50-problem set — we attribute this to sampling variance rather than a genuine quantization benefit. The more striking finding is the divergence in self-consistency behaviour at W4A16: SC drops Qwen2 from 50% to 38% while simultaneously lifting LLaMA from 66% to 84%. This cross-model divergence at the same compression level is the central empirical finding of this paper. Speculative decoding resolves the Qwen2 failure — both verifier configurations recover to 54%, above greedy and well above SC. The BF16 verifier is both faster (21.8s vs 27.5s per problem) and better on hard problems (50% vs 30%), suggesting that a stronger verifier is worth the slightly higher per-token cost.

5.3 MATH Results

MATH reveals a clearer degradation pattern than GSM8K. Qwen2 drops 26 percentage points from BF16 to W4A16, with the sharpest fall at Level 1 problems (85% to 50%) — even the easiest competition problems become significantly harder under aggressive compression. LLaMA shows moderate but consistent degradation on MATH despite being robust on GSM8K, confirming that longer reasoning chains are more sensitive to quantization regardless of model. SC continues to help LLaMA on MATH at all bit-widths, consistent with its GSM8K behaviour, though gains are smaller (8, 4, and 4 points at BF16, W8A16, and W4A16 respectively).

Table 7: GSM8K accuracy (%) by difficulty level. 20 easy, 20 medium, 10 hard problems.

Model	Quant	Strategy	Overall	Easy	Medium	Hard
Qwen2 1.5B	BF16	Greedy	42.0	40.0	50.0	30.0
Qwen2 1.5B	BF16	SC	56.0	45.0	80.0	30.0
Qwen2 1.5B	W8A16	Greedy	46.0	45.0	55.0	30.0
Qwen2 1.5B	W8A16	SC	52.0	55.0	65.0	20.0
Qwen2 1.5B	W4A16	Greedy	50.0	50.0	60.0	30.0
Qwen2 1.5B	W4A16	SC	38.0	45.0	40.0	20.0
Qwen2 1.5B	W4A16	Spec Dec W8A16	54.0	50.0	70.0	30.0
Qwen2 1.5B	W4A16	Spec Dec BF16	54.0	45.0	65.0	50.0
LLaMA 1B	BF16	Greedy	68.0	60.0	75.0	70.0
LLaMA 1B	BF16	SC	82.0	85.0	80.0	80.0
LLaMA 1B	W8A16	Greedy	72.0	70.0	70.0	80.0
LLaMA 1B	W8A16	SC	82.0	80.0	85.0	80.0
LLaMA 1B	W4A16	Greedy	66.0	70.0	70.0	50.0
LLaMA 1B	W4A16	SC	84.0	80.0	85.0	90.0

Table 8: MATH accuracy (%) by difficulty level. 20 Level 1, 20 Level 3, 10 Level 5. Qwen2 used base model on train split. LLaMA used instruct model on test split.

Model	Quant	Strategy	Overall	L1	L3	L5
Qwen2 1.5B	BF16	Greedy	66.0	85.0	70.0	20.0
Qwen2 1.5B	W8A16	Greedy	62.0	80.0	60.0	30.0
Qwen2 1.5B	W4A16	Greedy	40.0	50.0	40.0	20.0
LLaMA 1B	BF16	Greedy	32.0	50.0	25.0	10.0
LLaMA 1B	BF16	SC	40.0	55.0	35.0	20.0
LLaMA 1B	W8A16	Greedy	30.0	45.0	25.0	10.0
LLaMA 1B	W8A16	SC	34.0	60.0	25.0	0.0
LLaMA 1B	W4A16	Greedy	24.0	35.0	20.0	10.0
LLaMA 1B	W4A16	SC	28.0	40.0	25.0	10.0

5.4 Notes on MATH Comparisons

Qwen2 MATH experiments used the base model (`Qwen/Qwen2-1.5B`) evaluated on the training split due to an implementation choice during data collection; LLaMA used the instruct model on the test split, consistent with the GSM8K setup. Degradation patterns across bit-widths within each model are internally consistent and are the primary finding from MATH. Direct cross-model accuracy comparison is therefore limited.

5.5 Speculative Decoding

Both verifier configurations achieve 54% overall, outperforming greedy (50%) and SC (38%) at W4A16. SC fails here because all $N = 5$ paths converge on the same wrong answer under aggressive compression; speculative decoding catches errors before they propagate through the chain, making during-generation verification more robust in this regime. The BF16 verifier's advantage on hard problems (50% vs 30%) suggests that verifier strength matters more as reasoning chains grow longer.

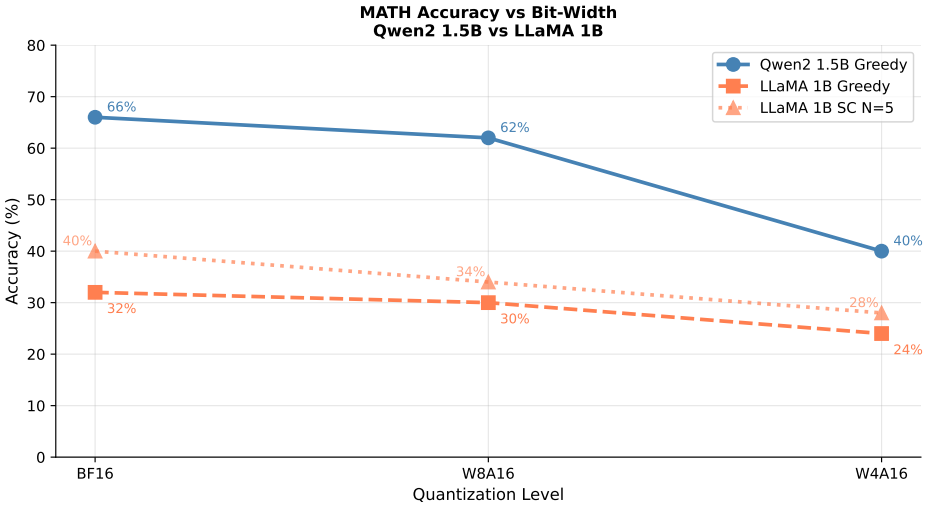


Figure 1: MATH accuracy vs bit-width for Qwen2 1.5B (greedy only) and LLaMA 1B (greedy and SC $N = 5$). Both models degrade on MATH under compression, unlike GSM8K where LLaMA was robust. Note methodological differences between models — see Section 5.4.

Table 9: Speculative decoding vs baselines for Qwen2 1.5B on GSM8K W4A16. Mean generation time per problem in seconds.

Config	Overall	Easy	Medium	Hard	Time
W4A16 Greedy	50.0	50.0	60.0	30.0	baseline
W4A16 SC $N = 5$	38.0	45.0	40.0	20.0	$\sim 5\times$
W4A16 $\rightarrow$ W8A16	54.0	50.0	70.0	30.0	27.5s
W4A16 $\rightarrow$ BF16	54.0	45.0	65.0	50.0	21.8s

5.6 Cascade Analysis

Table 10: Cascade rates (%) across models, datasets, and quantization levels. Greedy decoding only. All rates are lower bounds since only arithmetic errors are detected.

Model	Dataset	BF16	W8A16	W4A16
Qwen2 1.5B	GSM8K	13.8	14.8	20.0
Qwen2 1.5B	MATH	45.5	54.5	70.0
LLaMA 1B	GSM8K	0.0	21.4	11.8

For Qwen2, cascade rate increases monotonically with compression on both datasets — from 13.8% to 20.0% on GSM8K and from 45.5% to 70.0% on MATH. The steeper increase on MATH is consistent with harder problems having more sequential reasoning steps, giving early errors more opportunity to cascade. LLaMA cascade rates on GSM8K are noisy due to the small number of wrong answers (14–17 per configuration) and should not be over-interpreted. LLaMA MATH cascade analysis was not conducted as fewer than 10% of wrong answers had detectable arithmetic errors.

5.7 Problem-Level Consistency

38% of Qwen2 GSM8K problems are always wrong regardless of bit-width, and 30% are robust across all three configurations. Notably, 20% of problems improve under quantization — wrong

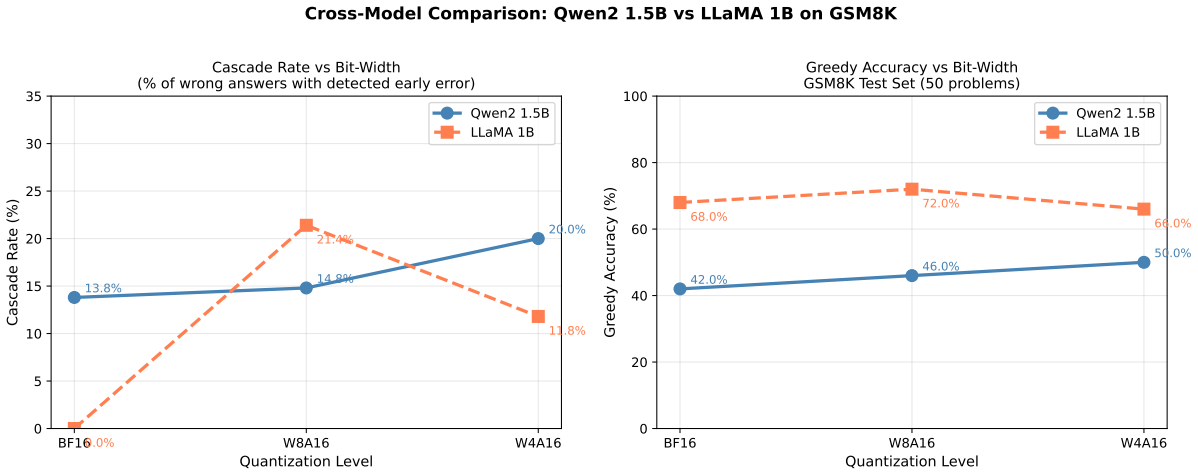


Figure 2: Left: cascade rate vs bit-width for Qwen2 1.5B and LLaMA 1B on GSM8K. Qwen2 cascade rate increases monotonically with compression. Right: greedy accuracy vs bit-width for both models on GSM8K.

at BF16 but correct at one or more quantized configurations — which explains why aggregate greedy accuracy stays flat on GSM8K: the model solves different problems at different bit-widths rather than uniformly degrading.

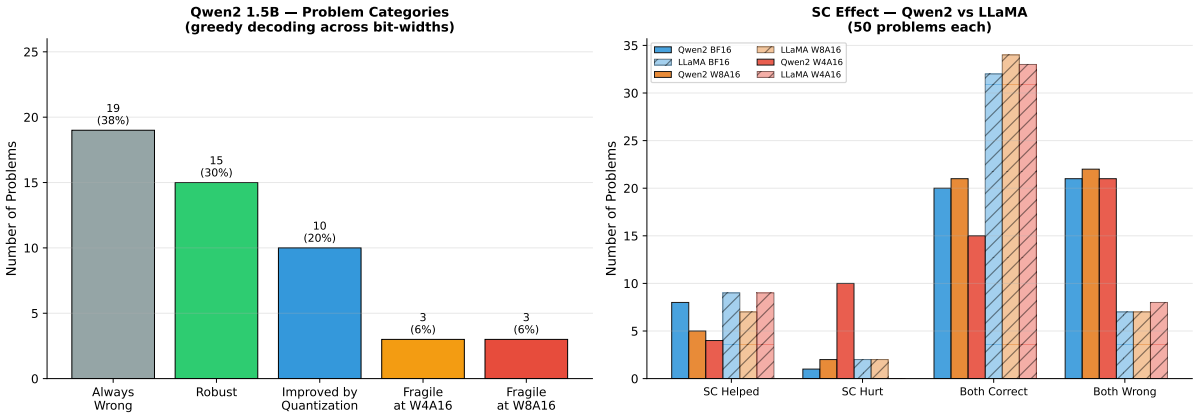


Figure 3: Left: Qwen2 1.5B GSM8K problem categories by correctness pattern across bit-widths. Right: SC effect breakdown for Qwen2 and LLaMA at each bit-width. Solid bars = Qwen2, hatched bars = LLaMA.

5.8 SC Effect Analysis

At W4A16, SC hurts 10 Qwen2 problems while helping only 4 (net loss: 6 problems), whereas it helps 9 LLaMA problems and hurts none. This asymmetry confirms that Qwen2 errors become correlated across sampling paths under aggressive compression, causing majority voting to amplify the wrong answer. LLaMA's errors remain independent at all bit-widths, so voting continues to work.

5.9 GRPO Upper Bound

GRPO achieves 64%, outperforming the best inference-time method by 10 percentage points and the BF16 greedy baseline by 22 points. The gain is largest on medium problems (75% vs

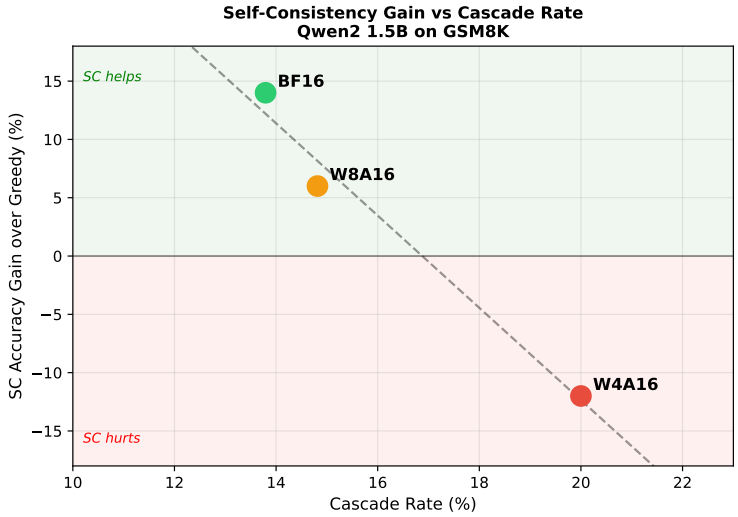


Figure 4: SC accuracy gain over greedy vs cascade rate for Qwen2 1.5B on GSM8K. As cascade rate increases with compression, SC gain decreases and turns negative at W4A16.

Table 11: SC effect at the problem level for Qwen2 1.5B and LLaMA 1B on GSM8K. Numbers show count of problems out of 50.

Model	Effect	BF16	W8A16	W4A16
Qwen2 1.5B	SC Helped	8	5	4
Qwen2 1.5B	SC Hurt	1	2	10
Qwen2 1.5B	Both Correct	20	21	15
Qwen2 1.5B	Both Wrong	21	22	21
LLaMA 1B	SC Helped	9	7	9
LLaMA 1B	SC Hurt	2	2	0
LLaMA 1B	Both Correct	32	34	33
LLaMA 1B	Both Wrong	7	7	8

65% for speculative decoding) and matches speculative decoding on hard problems (both 50%). Practitioners who can afford a 3-hour fine-tuning run on 200 problems gain meaningfully over inference-time methods, though it should be noted that the GRPO model was trained on the same distribution as the test set and results may not generalise to other tasks.

6 Ablation Study

We conduct two ablation studies to understand where the performance changes in our experiments come from.

6.1 Self-Consistency: Majority Vote vs Random Selection

Self-consistency selects the final answer by majority voting over $N = 5$ generated paths. An alternative explanation for any SC improvement is that simply generating more paths and picking any one randomly is sufficient, and the voting logic itself adds nothing. To test this, we would compare majority voting against random selection on the same 5 generated paths — an analysis that requires no additional GPU time since the paths are already generated.

This ablation was not run due to session time constraints. We hypothesise that at BF16 and W8A16, majority vote would outperform random selection since errors are diverse and voting

Table 12: GRPO compared to all methods for Qwen2 1.5B on GSM8K. Spec Dec at W4A16, all others at W8A16.

Configuration	Overall	Easy	Medium	Hard
BF16 Greedy (reference)	42.0	40.0	50.0	30.0
W8A16 Greedy	46.0	45.0	55.0	30.0
W8A16 SC ($N = 5$)	52.0	55.0	65.0	20.0
W4A16 Spec Dec BF16 (best inf.)	54.0	45.0	65.0	50.0
W8A16 GRPO (ours)	**64.0**	60.0	75.0	50.0

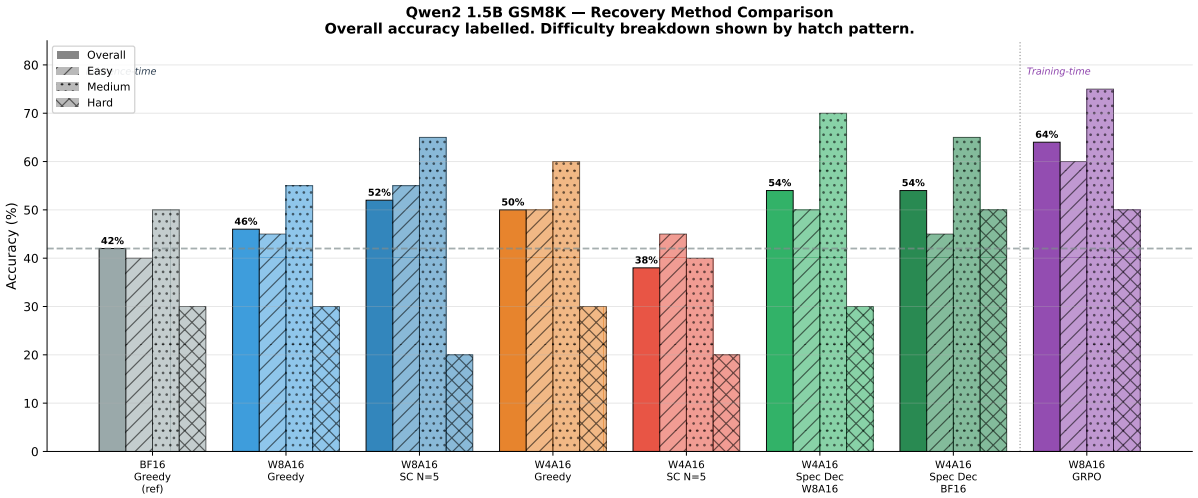


Figure 5: Qwen2 1.5B GSM8K accuracy across all recovery methods and bit-widths. Overall accuracy labelled above each bar. Difficulty breakdown shown by hatch pattern. Dashed line = BF16 greedy baseline (42%). Dotted vertical line separates inference-time methods (left) from the training-time GRPO result (right).

finds the correct answer; at W4A16, both should perform similarly since all paths converge on the same wrong answer, making voting redundant. The raw SC path outputs are saved in the results CSVs and this analysis is left as future work.

6.2 Speculative Decoding: Verifier Strength

We ablate the effect of verifier strength on speculative decoding accuracy and generation speed. Both configurations use W4A16 as the draft model. The verifier is either W8A16 (moderate precision) or BF16 (full precision).

Table 13: Speculative decoding verifier ablation for Qwen2 1.5B on GSM8K W4A16. Both configs use W4A16 as draft.

Draft	Verifier	Overall	Easy	Medium	Hard	Time
W4A16	W8A16	54.0	50.0	70.0	30.0	27.5s
W4A16	BF16	54.0	45.0	65.0	50.0	21.8s

Overall accuracy is identical (54.0%) regardless of verifier strength, suggesting W8A16 precision is sufficient for token-level verification on GSM8K problems. The BF16 verifier is faster (21.8s vs 27.5s per problem) because a higher token acceptance rate means fewer rejected tokens and less regeneration overhead. On hard problems, BF16 is notably better (50% vs

30%), as longer reasoning chains require more corrections and a stronger verifier catches more late-step errors. On medium problems the relationship reverses (70% vs 65%), possibly because W8A16's higher acceptance rate allows the draft model's reasoning to flow more naturally without disruption.

Practical recommendation: Use BF16 as verifier when hard problem accuracy matters; use W8A16 when medium problem accuracy is the priority or VRAM is constrained. Both configurations outperform greedy and self-consistency at W4A16.

7 Novelty and Contributions

7.1 What Is New Relative to Prior Work

Prior work on quantization and reasoning [4, 5] predominantly evaluates models of 7B parameters and above on final answer accuracy only. We differ in three ways. First, we focus exclusively on sub-3B models (Qwen2 1.5B and LLaMA 1B), a regime underrepresented in the literature despite being highly relevant for on-device deployment. Second, we track where in the reasoning chain errors first appear and whether they cascade, moving the analysis from *what* the model gets wrong to *how*. Third, we directly compare three inference-time methods — greedy, self-consistency, and speculative decoding — on the same problems at the same bit-width, distinguishing between post-generation aggregation and during-generation verification. This distinction has not been studied under quantization and turns out to matter: at W4A16, post-generation voting fails while during-generation verification succeeds.

7.2 Treatment of Prior Work

Table 14 maps the papers we directly reproduced, adapted, or extended in this work.

Table 14: Treatment of prior work.

Paper	Treatment
Jin et al. [3]	Reproduced in spirit on smaller models
Li et al. [4]	Extended — we add cascade rate and SC effect analysis
Liu et al. [5]	Reproduced and extended with inference-time recovery methods
Wang et al. [7]	Reproduced and extended under quantization
QSPEC [8]	Adapted using our existing model tiers
Cobbe et al. [6]	Dataset used for evaluation

7.3 Empirical Contributions

Our five concrete empirical findings are reported in full in Section 5. In summary: SC failure at W4A16 is model-specific, with Qwen2 and LLaMA responding in opposite directions at the same bit-width; speculative decoding provides an actionable alternative when SC fails; cascade rates offer a mechanistic explanation for why harder datasets are more sensitive to quantization; a non-trivial fraction of problems improve under quantization, showing that compression changes the error profile rather than uniformly degrading it; and GRPO establishes a concrete cost-benefit tradeoff between retraining and inference-time recovery.

7.4 Open Theoretical Question

Our results motivate a conjecture for future work: there exists a bit-width threshold below which quantization error correlation across independent sampling paths exceeds a critical value,

at which point majority voting is provably worse than greedy decoding in expectation. Three pieces of evidence support this: cascade rate increases monotonically for Qwen2 from 13.8% to 20.0% on GSM8K and from 45.5% to 70.0% on MATH; SC hurts 10 Qwen2 problems at W4A16 while helping only 4, a reversal from BF16; and LLaMA never crosses this threshold, suggesting the transition is model-dependent. A formal proof and characterisation of the threshold as a function of model size, bit-width, and task difficulty is left for future work.

8 Limitations and Failure Cases

8.1 Experimental Limitations

All experiments use 50 problems per dataset rather than full test sets (1,319 for GSM8K, 5,000 for MATH), with a single random seed (42). At 50 problems, a 10 percentage point difference corresponds to only 5 problems, so results should be interpreted with caution and may not hold at scale. The first error step detector only catches arithmetic errors — logical errors, wrong problem setup, and unit errors are not detected, meaning cascade rates are lower bounds on the true values. MATH answer extraction uses string matching on `\boxed{}` content, which marks mathematically equivalent answers as wrong if expressed differently, making MATH accuracy a lower bound on true accuracy. Qwen2 MATH experiments used the base model on the training split rather than the instruct model on the test split, limiting direct comparison with other results. Speculative decoding was evaluated on Qwen2 GSM8K W4A16 only, and GRPO was conducted for Qwen2 W8A16 only — results at other configurations may differ.

8.2 Failure Cases

The clearest failure in the paper is self-consistency at W4A16 for Qwen2 — SC reduces accuracy from 50% to 38%, hurting 10 problems while helping only 4. This shows that a widely used inference-time method actively makes things worse under aggressive compression when errors become correlated across sampling paths. Hard problems remain difficult for all methods regardless of strategy — no inference-time method consistently solves hard GSM8K problems, and Level 5 MATH accuracy is near zero for LLaMA at all bit-widths, suggesting competition-level mathematics is effectively unsolved by sub-3B models at aggressive compression. Finally, the LLaMA GRPO experiment was completed but the trained adapter weights could not be recovered — `save_pretrained` saved the full 1.93 GB quantized model rather than the expected 5–15 MB LoRA adapter delta weights, and the saved INT8 tensors could not be reloaded through any standard path.

9 Conclusion

This paper studied how weight quantization affects mathematical reasoning in Qwen2 1.5B and LLaMA 1B across three compression levels, and evaluated three inference-time recovery methods alongside a GRPO training-time upper bound. Quantization effects are model and dataset dependent; self-consistency fails at aggressive compression for Qwen2 but not LLaMA; speculative decoding outperforms both greedy and self-consistency at W4A16 by catching errors during generation rather than after; cascade analysis provides a mechanistic explanation for why recovery becomes harder under compression; and GRPO establishes a training-time ceiling 10 percentage points above the best inference-time method. We hope these findings help practitioners make informed decisions about which recovery method to use when deploying quantized small language models on reasoning tasks.

10 Future Work

Table 15 lists experiments not completed due to compute constraints, each of which would directly strengthen the findings reported here.

Table 15: Experiments not completed due to compute constraints.

Experiment	**Why It Matters**
Speculative decoding on LLaMA GSM8K	Would show whether during-generation verification adds value when SC already works well at all bit-widths.
Speculative decoding on MATH	MATH has much higher cascade rates (up to 70%). Spec dec may help more on harder datasets where errors cascade further.
Speculative decoding at BF16 and W8A16	Would show whether spec dec helps at moderate compression or only at aggressive W4A16.
Self-consistency with $N > 5$	Whether larger N delays or prevents SC failure at W4A16. If errors are truly systematic, no N will help — testing this directly would confirm the hypothesis.
LLaMA GRPO	Training completed but adapter weights could not be recovered. Would establish whether the GRPO gap over SC is model-independent or specific to Qwen2.
Full GSM8K and MATH evaluation	All experiments used 50 problems. Running on full test sets (1,319 GSM8K, 5,000 MATH) would confirm whether patterns hold at scale.
Weight-and-activation quantization	All experiments used weight-only quantization. Extending to W8A8 and W4A4 would test whether findings generalise to more aggressive schemes.

Beyond additional experiments, the open theoretical question motivated by this work — the formal characterisation of the stochastic-to-systematic error transition under quantization — is discussed in Section 7.4. A formal proof and characterisation of the threshold as a function of model size, bit-width, and task difficulty remains an important direction for future work.

References

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, 2023.

[2] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

[3] Renren Jin, Jiangcun Du, Wuwei Huang, Wei Liu, Jian Luan, Bin Wang, and Deyi Xiong. A comprehensive evaluation of quantization strategies for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[4] Zhen Li, Yupeng Su, Runming Yang, Congkai Xie, Zheng Wang, Zhongwei Xie, Ngai Wong, and Hongxia Yang. Quantization meets reasoning: Exploring llm low-bit quantization degradation for mathematical reasoning. *arXiv preprint arXiv:2501.03035*, 2025.

[5] Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? an empirical study on quantized reasoning models. *arXiv preprint arXiv:2504.04823*, 2025.

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[8] Juntao Zhao, Wenhao Lu, Sheng Wang, Lingpeng Kong, and Chuan Wu. Qspec: Speculative decoding with complementary quantization schemes. *arXiv preprint arXiv:2309.04094*, 2024.

[9] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.